

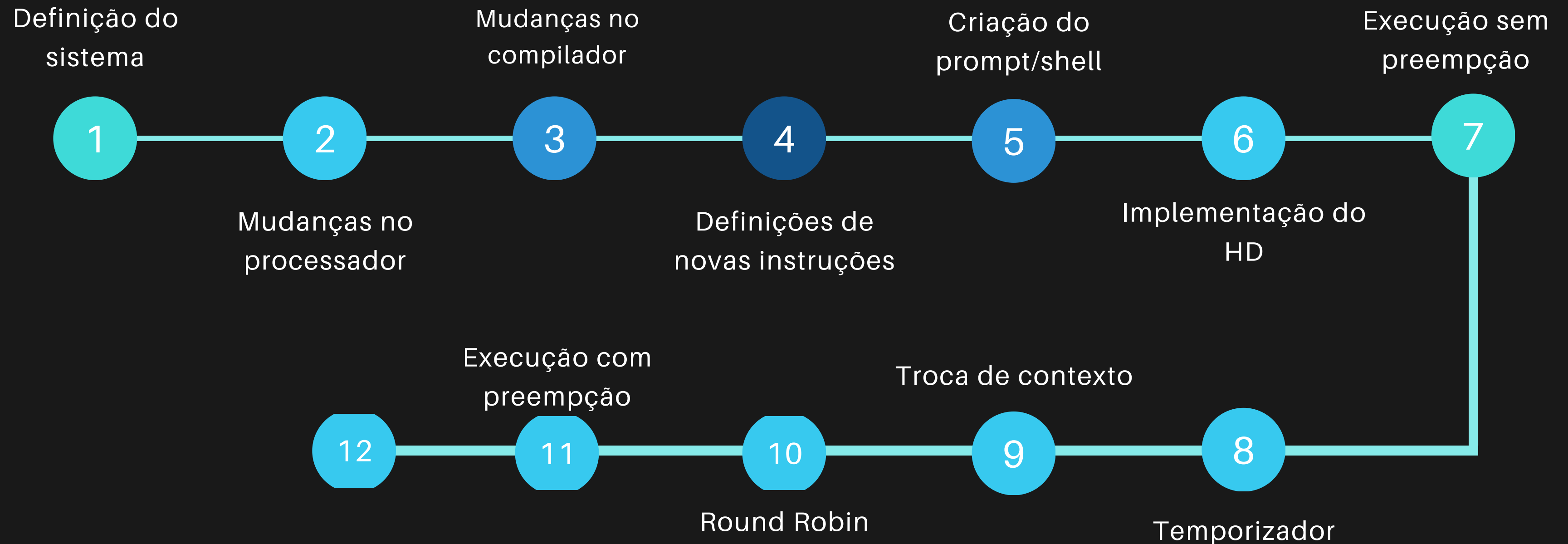
Projeto Sistemas Operacionais

Aluno: Bruno Ogata

Prof: Tiago Oliveira

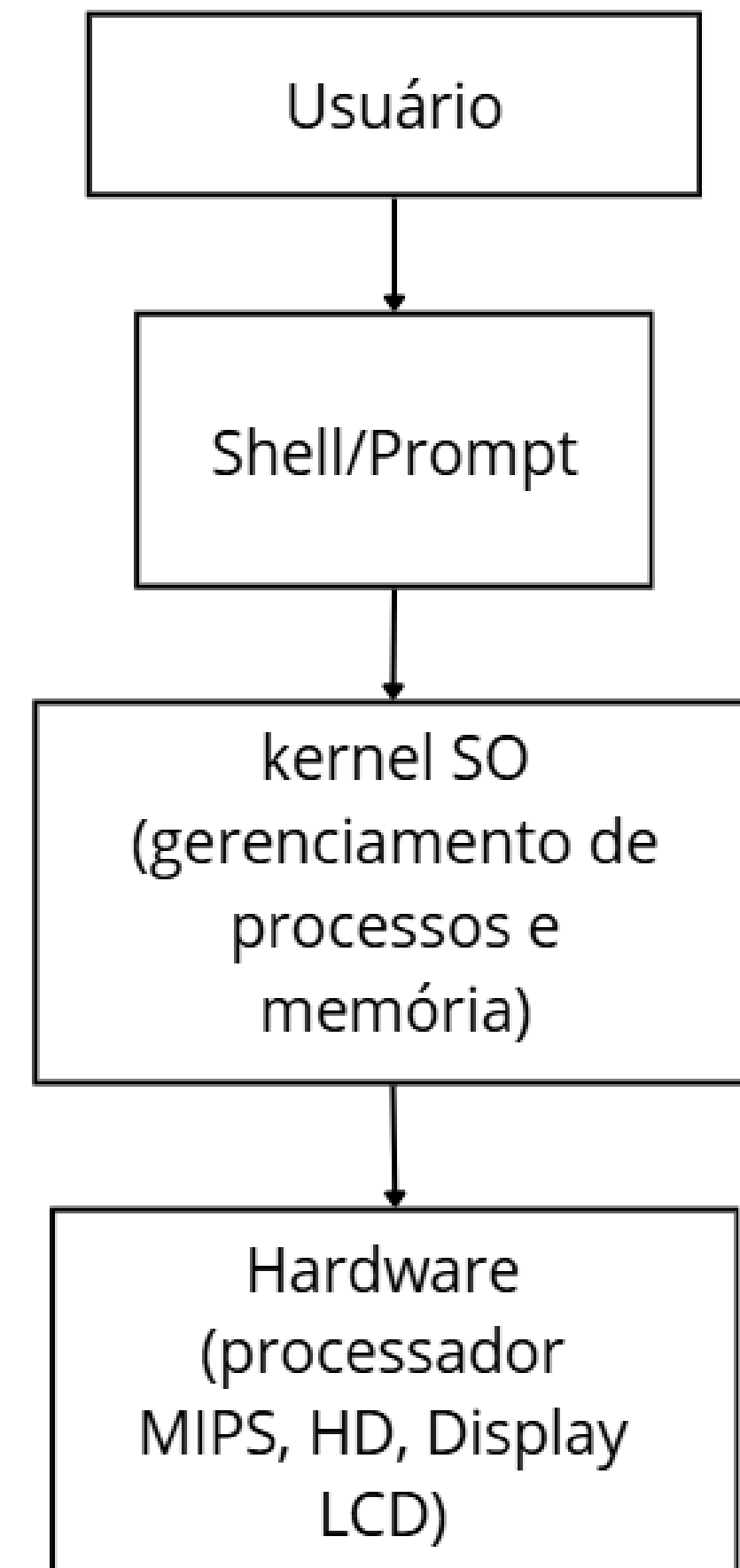
Projeto de Sistemas Operacionais

Fluxograma da apresentação



Definição do sistema

- 1 Interação via prompt/shell para entrada de comandos
- 2 Gerenciamento de processos com execução sem e com preempção
- 3 Escalonador Round Robin implementado
- 4 HD simulado para armazenamento dos programas



Mudanças no processador

- 1 Criação de módulos e instrução para display LCD
- 2 mudança na memória de instruções ROM → RAM
- 3 Implementação de HD simulado e instrução para carregar do hd para ram de instruções

- 4 Criação de módulos para auxiliar no gerenciamento de memória
- 5 Criação de temporizador e gerenciamento de interrupções
- 6 Implementação de novas instruções para realizar a troca de contexto

Mudanças no compilador

- 1 Criação de funções especiais para auxiliar no desenvolvimento do SO
 - 1.1 Função para salvar o contexto de um processo na MD
 - 1.2 Função para recuperar o contexto de um processo da MD
 - 1.3 Função para contar quantos halts foram executados
 - 1.4 Função para limpar a MD

Definição de novas instruções

1 Set_LCD → instrução para exibir mensagem no display

3 Load_Program → instrução que carrega um programa (por inteiro) do HD para a memória

2 Change_Offset → instrução que altera o offset nos endereços da MD

4 Set_Quantum → Inicia contagem do quantum (instrução)

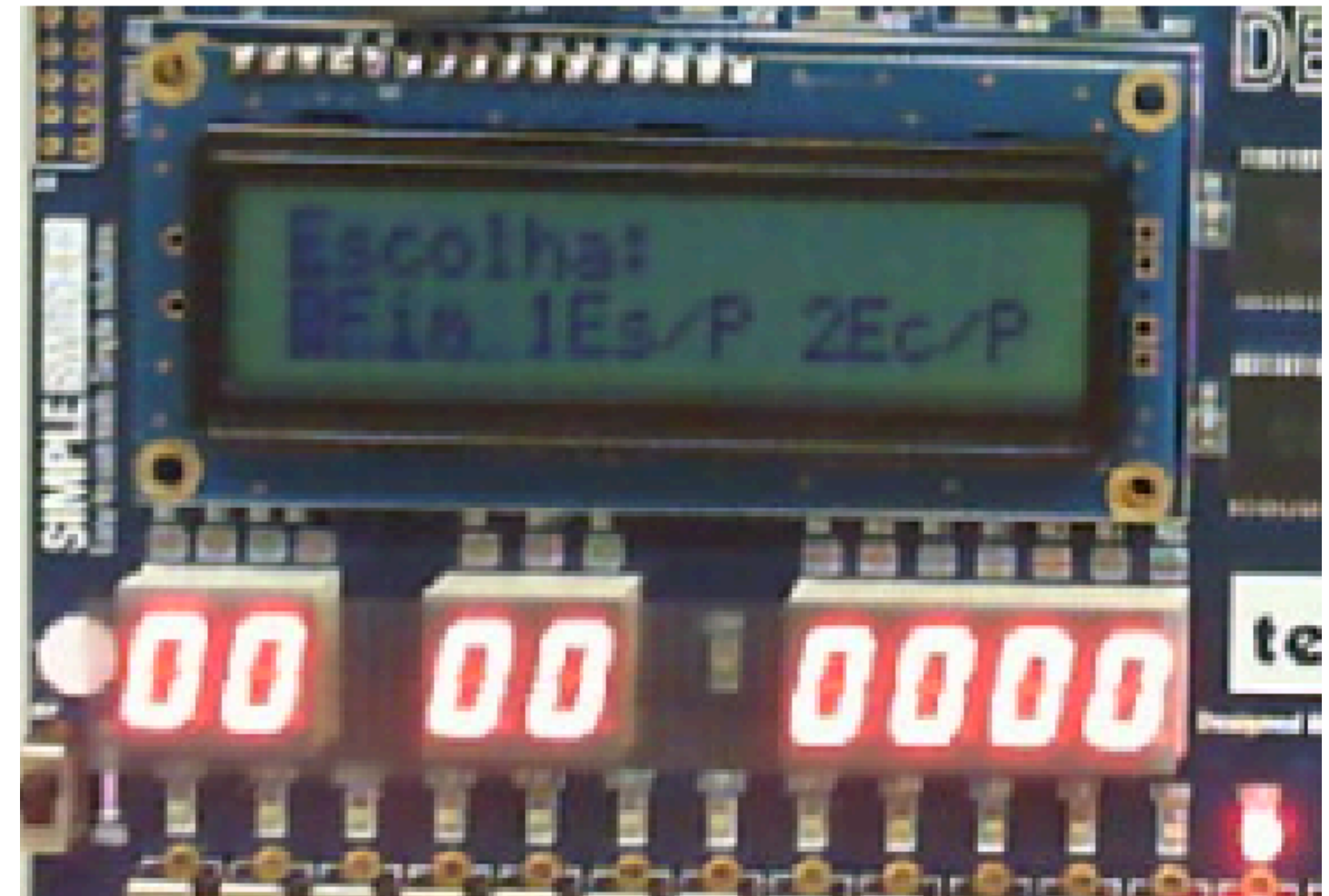
5 jalr → faz o jump para um registrador e salva o endereço de retorno

Criação do prompt

Código em C-

```
int menu(void)
{
    int escolha;
    escolha = 0;
    setLcd(10);
    escolha = input();
    return escolha;
}
```

Visualização no FPGA



Implementação do HD - módulo do HD

Código em Verilog

```

module Hard_Disk (
    input Clock,
    input [11:0] addr,          // Endereço dentro do programa
    output reg [31:0] instrucao // Instrução lida
);

reg [31:0] memoria_hd [0:2047];

initial begin
    // Programa 1 - GCD
    memoria_hd[200] = 32'b000111000000000000000000000001001101; // jump 77
    memoria_hd[201] = 32'b00010011101111110000000000000001; // sw $fp $ra 1
    memoria_hd[202] = 32'b00000111110111110000000000000001; // addi $sp $sp 1
    memoria_hd[203] = 32'b00000111110111110000000000000001; // addi $sp $sp 1
    memoria_hd[204] = 32'b00000111110111110000000000000001; // addi $sp $sp 1
    memoria_hd[205] = 32'b00001111101000010000000000000010; // lw $fp $t1 2
    memoria_hd[206] = 32'b00001111101000100000000000000011; // lw $fp $t2 3
    memoria_hd[207] = 32'b00000111110111110000000000000001; // addi $sp $sp 1
    memoria_hd[208] = 32'b00001111101001000000000000000100; // lw $fp $t4 4
    memoria_hd[209] = 32'b00000100100001000000000000000000; // addi $t4 $t4 0
    memoria_hd[210] = 32'b00010011101001000000000000000100; // sw $fp $t4 4
    memoria_hd[211] = 32'b00000000001000100010100000100101; // sge $t5 $t1 $t2
    memoria_hd[212] = 32'b000101000000010100000000000011011; // beq $t5 $zero 27
    memoria_hd[213] = 32'b00001111101001100000000000000010; // lw $fp $t6 2
    memoria_hd[214] = 32'b00001111101001110000000000000010; // lw $fp $t7 2
    memoria_hd[215] = 32'b00001111101010000000000000000011; // lw $fp $t8 3
    memoria_hd[216] = 32'b000000000111010000100100000000001; // sub $t9 $t7 $t8
    memoria_hd[217] = 32'b00000101001001100000000000000000; // addi $t6 $t9 0
    memoria_hd[218] = 32'b00010011101001100000000000000010; // sw $fp $t6 2
    memoria_hd[219] = 32'b00001111101010100000000000000100; // lw $fp $t10 4
    memoria_hd[220] = 32'b00001111101010110000000000000100; // lw $fp $t11 4
    memoria_hd[221] = 32'b00000100000110110000000000000001; // addi $aux $zero 1
    memoria_hd[222] = 32'b00000001011110110110000000000000; // add $t12 $t11 $aux
    memoria_hd[223] = 32'b00000101100010100000000000000000; // addi $t10 $t12 0
    memoria_hd[224] = 32'b00010011101010100000000000000100; // sw $fp $t10 4
    memoria_hd[225] = 32'b00000101100000010000000000000000; // addi $t1 $t12 0
    memoria_hd[226] = 32'b00011100000000000000000000001011; // jump 11
    memoria_hd[227] = 32'b00001111101011010000000000000100; // lw $fp $t13 4
    memoria_hd[228] = 32'b00001011011011100000000000000000; // addi $t11 $t13 0
    memoria_hd[229] = 32'b00000101100001000000000000000000; // addi $t12 $t11 0
    memoria_hd[230] = 32'b00000101100001000000000000000000; // addi $t12 $t11 0
    memoria_hd[231] = 32'b00000101100001000000000000000000; // addi $t12 $t11 0
    memoria_hd[232] = 32'b00000101100001000000000000000000; // addi $t12 $t11 0
    memoria_hd[233] = 32'b00000101100001000000000000000000; // addi $t12 $t11 0
    memoria_hd[234] = 32'b00000101100001000000000000000000; // addi $t12 $t11 0
    memoria_hd[235] = 32'b00000101100001000000000000000000; // addi $t12 $t11 0
    memoria_hd[236] = 32'b00000101100001000000000000000000; // addi $t12 $t11 0
    memoria_hd[237] = 32'b00000101100001000000000000000000; // addi $t12 $t11 0
    memoria_hd[238] = 32'b00000101100001000000000000000000; // addi $t12 $t11 0
    memoria_hd[239] = 32'b00000101100001000000000000000000; // addi $t12 $t11 0
    memoria_hd[240] = 32'b00000101100001000000000000000000; // addi $t12 $t11 0
    memoria_hd[241] = 32'b00000101100001000000000000000000; // addi $t12 $t11 0
    memoria_hd[242] = 32'b00000101100001000000000000000000; // addi $t12 $t11 0
    memoria_hd[243] = 32'b00000101100001000000000000000000; // addi $t12 $t11 0
    memoria_hd[244] = 32'b00000101100001000000000000000000; // addi $t12 $t11 0
    memoria_hd[245] = 32'b00000101100001000000000000000000; // addi $t12 $t11 0
    memoria_hd[246] = 32'b00000101100001000000000000000000; // addi $t12 $t11 0
    memoria_hd[247] = 32'b00000101100001000000000000000000; // addi $t12 $t11 0
    memoria_hd[248] = 32'b00000101100001000000000000000000; // addi $t12 $t11 0
    memoria_hd[249] = 32'b00000101100001000000000000000000; // addi $t12 $t11 0
    memoria_hd[250] = 32'b00000101100001000000000000000000; // addi $t12 $t11 0
    memoria_hd[251] = 32'b00000101100001000000000000000000; // addi $t12 $t11 0
    memoria_hd[252] = 32'b00000101100001000000000000000000; // addi $t12 $t11 0
    memoria_hd[253] = 32'b00000101100001000000000000000000; // addi $t12 $t11 0
    memoria_hd[254] = 32'b00000101100001000000000000000000; // addi $t12 $t11 0
    memoria_hd[255] = 32'b00000101100001000000000000000000; // addi $t12 $t11 0
    memoria_hd[256] = 32'b00000101100001000000000000000000; // addi $t12 $t11 0
    memoria_hd[257] = 32'b00000101100001000000000000000000; // addi $t12 $t11 0
    memoria_hd[258] = 32'b00000101100001000000000000000000; // addi $t12 $t11 0
    memoria_hd[259] = 32'b00000101100001000000000000000000; // addi $t12 $t11 0
    memoria_hd[260] = 32'b00000101100001000000000000000000; // addi $t12 $t11 0
    memoria_hd[261] = 32'b00000101100001000000000000000000; // addi $t12 $t11 0
    memoria_hd[262] = 32'b00000101100001000000000000000000; // addi $t12 $t11 0
    memoria_hd[263] = 32'b00000101100001000000000000000000; // addi $t12 $t11 0
    memoria_hd[264] = 32'b00000101100001000000000000000000; // addi $t12 $t11 0
    memoria_hd[265] = 32'b00000101100001000000000000000000; // addi $t12 $t11 0
    memoria_hd[266] = 32'b00000101100001000000000000000000; // addi $t12 $t11 0
    memoria_hd[267] = 32'b00000101100001000000000000000000; // addi $t12 $t11 0
    memoria_hd[268] = 32'b0000010110000
```

```

initial begin
    memoria_hd[1436] = 32'b00000110000011010000000000000000; // addi $t13 $t16 0
    memoria_hd[1437] = 32'b00010011101011010000000000000011; // sw $fp $t13 3
    memoria_hd[1438] = 32'b000111000000000000000000000111001; // jump 57
    memoria_hd[1439] = 32'b00000100000110110000000000000011; // addi $aux $zero 3
    memoria_hd[1440] = 32'b00000000001110111000100000000001; // sub $t17 $t1 $aux
    memoria_hd[1441] = 32'b000110000001000100000000000110001; // bne $t17 $zero 49
    memoria_hd[1442] = 32'b00001111101100100000000000000011; // lw $fp $t18 3
    memoria_hd[1443] = 32'b00001111101100110000000000000001; // lw $fp $t19 1
    memoria_hd[1444] = 32'b00001111101101000000000000000010; // lw $fp $t20 2
    memoria_hd[1445] = 32'b00000010011101001010100000000010; // mult $t21 $t19 $t20
    memoria_hd[1446] = 32'b00000110101100100000000000000000; // addi $t18 $t21 0
    memoria_hd[1447] = 32'b00010011101100100000000000000011; // sw $fp $t18 3
    memoria_hd[1448] = 32'b000111000000000000000000000111000; // jump 56
    memoria_hd[1449] = 32'b00001111101101100000000000000011; // lw $fp $t22 3
    memoria_hd[1450] = 32'b00001111101101110000000000000001; // lw $fp $t23 1
    memoria_hd[1451] = 32'b00001111101110000000000000000010; // lw $fp $t24 2
    memoria_hd[1452] = 32'b00000010111110001100100000000011; // div $t25 $t23 $t24
    memoria_hd[1453] = 32'b00000111001101100000000000000000; // addi $t22 $t25 0
    memoria_hd[1454] = 32'b00010011101101100000000000000011; // sw $fp $t22 3
    memoria_hd[1455] = 32'b000111000000000000000000000111000; // jump 56
    memoria_hd[1456] = 32'b000111000000000000000000000111001; // jump 57
    memoria_hd[1457] = 32'b000111000000000000000000000111010; // jump 58
    memoria_hd[1458] = 32'b00001111101110100000000000000011; // lw $fp $t26 3
    memoria_hd[1459] = 32'b00101011010111111111111111111111; // output $t26
    memoria_hd[1460] = 32'b11111000000000000000000000000000; // halt
end

always @(posedge Clock) begin
    instracao <= memoria_hd[addr];
end
endmodule

```

Implementação do HD - módulo de controle

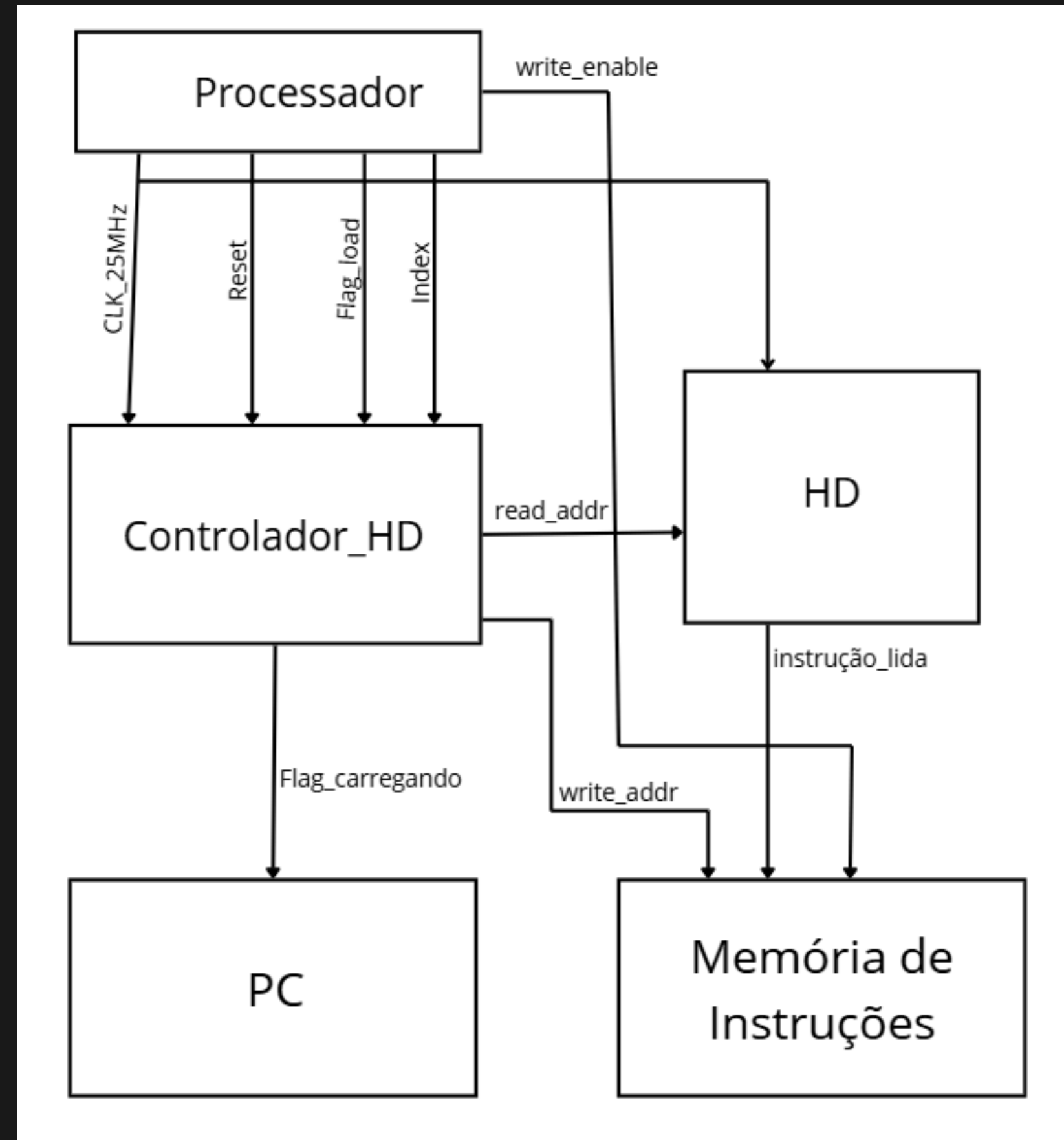
Código em Verilog

```
module Controlador_HD
#(parameter ADDR_WIDTH = 12, parameter PROG_SIZE = 150, parameter RAM_SIZE = 700)
(
    input Clock,                // Clock do sistema
    input Reset,                // Sinal de reset
    input Load_from_HD,        // Sinal para controle do Load do HD
    input [ADDR_WIDTH-1:0] indice_programa, // Índice do programa a ser carregado
    output reg carregando,      // Sinal indicando se o carregamento está em andamento
    output reg [ADDR_WIDTH-1:0] endereco_HD, // Endereço no HD para leitura
    output reg [ADDR_WIDTH-1:0] endereco_RAM // Endereço na RAM para escrita
);

// Lógica de controle
always @(posedge Clock) begin
    if (Reset) begin
        carregando <= 0;
        endereco_HD <= 0;
        endereco_RAM <= 0;
    end else if (carregando == 1) begin
        if (endereco_RAM < RAM_SIZE) begin
            endereco_HD <= endereco_HD + 1;
            endereco_RAM <= endereco_RAM + 1;
        end else begin
            carregando <= 0; // Finalizou o carregamento
            //count <= 0;
        end
    end else if (Load_from_HD == 1) begin
        carregando <= 1;
        //count <= 1;
        endereco_HD <= (indice_programa * PROG_SIZE) + 200; // Offset do programa no HD
        endereco_RAM <= 550; // Carregar a partir do índice 550 na RAM
    end
end
endmodule
```

Implementação do HD - integração dos módulos

IMPLEMENTAÇÃO DO HD - INTEGRAÇÃO DOS MÓDULOS



Execução sem preempção

Código em C-

```
if (modo == 1) {  
    setLcd(15);  
    program = input();  
    setLcd(program + 20);  
  
    loadProgram(program);  
    setLcd(program);  
  
    saveRegs();  
    changeOffset(program);  
    restoreRegs();  
  
    setOffset();  
  
    saveRegs();  
    changeOffset(0);  
    restoreRegs();  
}
```

Temporizador

Código em Verilog

```
module Temporizador (  
    input Clock,           // Clock do sistema  
    input Reset,           // Reset do sistema  
    input Quantum_flag,    // Sinal externo para reset do quantum_end  
    input [1:0] Entrada,  
    input Halt,  
    output reg quantum_end // Indica quando o quantum termina  
);  
  
parameter LIMITE_QUANTUM = 16; // Limite do quantum  
  
reg [7:0] contador; // Contador para o quantum  
initial begin  
    contador <= 0;  
    quantum_end <= 0;  
end  
  
always @(posedge Clock) begin  
    if (Reset || Halt) begin  
        contador <= 0;  
        quantum_end <= 0;  
    end else  
        if (Quantum_flag == 1 && Entrada != 2'b01) begin  
            if (contador < LIMITE_QUANTUM - 1) begin  
                // Incrementa o contador enquanto não atingir o limite do quantum  
                contador <= contador + 1;  
                quantum_end <= 0;  
            end else begin  
                // Quando o limite do quantum é atingido  
                contador <= 0;  
                quantum_end <= 1;  
            end  
        end  
    end  
end  
  
endmodule
```

Troca de contexto - função para salvar registradores

Código em C

```
}else if(strcmp(quads-&gtquad->var1, "saveRegs") == 0){  
    instrucao_insert(instrucoes, "sw", "$zero", "$t1", "21", I);  
    instrucao_insert(instrucoes, "sw", "$zero", "$t2", "22", I);  
    instrucao_insert(instrucoes, "sw", "$zero", "$t3", "23", I);  
    instrucao_insert(instrucoes, "sw", "$zero", "$t4", "24", I);  
    instrucao_insert(instrucoes, "sw", "$zero", "$t5", "25", I);  
    instrucao_insert(instrucoes, "sw", "$zero", "$t6", "26", I);  
    instrucao_insert(instrucoes, "sw", "$zero", "$t7", "27", I);  
    instrucao_insert(instrucoes, "sw", "$zero", "$t8", "28", I);  
    instrucao_insert(instrucoes, "sw", "$zero", "$t9", "29", I);  
    instrucao_insert(instrucoes, "sw", "$zero", "$t10", "30", I);  
    instrucao_insert(instrucoes, "sw", "$zero", "$t11", "31", I);  
    instrucao_insert(instrucoes, "sw", "$zero", "$t12", "32", I);  
    instrucao_insert(instrucoes, "sw", "$zero", "$t13", "33", I);  
    instrucao_insert(instrucoes, "sw", "$zero", "$t14", "34", I);  
    instrucao_insert(instrucoes, "sw", "$zero", "$t15", "35", I);  
    instrucao_insert(instrucoes, "sw", "$zero", "$t16", "36", I);  
    instrucao_insert(instrucoes, "sw", "$zero", "$t17", "37", I);  
    instrucao_insert(instrucoes, "sw", "$zero", "$t18", "38", I);  
    instrucao_insert(instrucoes, "sw", "$zero", "$t19", "39", I);  
    instrucao_insert(instrucoes, "sw", "$zero", "$t20", "40", I);  
    instrucao_insert(instrucoes, "sw", "$zero", "$t21", "41", I);  
    instrucao_insert(instrucoes, "sw", "$zero", "$t22", "42", I);  
    instrucao_insert(instrucoes, "sw", "$zero", "$t23", "43", I);  
    instrucao_insert(instrucoes, "sw", "$zero", "$hc", "44", I);  
    instrucao_insert(instrucoes, "sw", "$zero", "$spc", "45", I);  
    instrucao_insert(instrucoes, "sw", "$zero", "$pc", "46", I);  
    instrucao_insert(instrucoes, "sw", "$zero", "$aux", "47", I);  
    instrucao_insert(instrucoes, "sw", "$zero", "$gp", "48", I);  
    instrucao_insert(instrucoes, "sw", "$zero", "$fp", "49", I);  
    instrucao_insert(instrucoes, "sw", "$zero", "$sp", "50", I);  
    instrucao_insert(instrucoes, "sw", "$zero", "$ra", "51", I);  
    contInstrucoes = contInstrucoes + 31;  
}
```

Troca de contexto - função para restaurar registradores

Código em C

```
}else if(strcmp(quads->quad->var1, "restoreRegs") == 0){
    instrucao_insert(instrucoes, "lw", "$zero", "$t1", "21", I);
    instrucao_insert(instrucoes, "lw", "$zero", "$t2", "22", I);
    instrucao_insert(instrucoes, "lw", "$zero", "$t3", "23", I);
    instrucao_insert(instrucoes, "lw", "$zero", "$t4", "24", I);
    instrucao_insert(instrucoes, "lw", "$zero", "$t5", "25", I);
    instrucao_insert(instrucoes, "lw", "$zero", "$t6", "26", I);
    instrucao_insert(instrucoes, "lw", "$zero", "$t7", "27", I);
    instrucao_insert(instrucoes, "lw", "$zero", "$t8", "28", I);
    instrucao_insert(instrucoes, "lw", "$zero", "$t9", "29", I);
    instrucao_insert(instrucoes, "lw", "$zero", "$t10", "30", I);
    instrucao_insert(instrucoes, "lw", "$zero", "$t11", "31", I);
    instrucao_insert(instrucoes, "lw", "$zero", "$t12", "32", I);
    instrucao_insert(instrucoes, "lw", "$zero", "$t13", "33", I);
    instrucao_insert(instrucoes, "lw", "$zero", "$t14", "34", I);
    instrucao_insert(instrucoes, "lw", "$zero", "$t15", "35", I);
    instrucao_insert(instrucoes, "lw", "$zero", "$t16", "36", I);
    instrucao_insert(instrucoes, "lw", "$zero", "$t17", "37", I);
    instrucao_insert(instrucoes, "lw", "$zero", "$t18", "38", I);
    instrucao_insert(instrucoes, "lw", "$zero", "$t19", "39", I);
    instrucao_insert(instrucoes, "lw", "$zero", "$t20", "40", I);
    instrucao_insert(instrucoes, "lw", "$zero", "$t21", "41", I);
    instrucao_insert(instrucoes, "lw", "$zero", "$t22", "42", I);
    instrucao_insert(instrucoes, "lw", "$zero", "$t23", "43", I);
    instrucao_insert(instrucoes, "lw", "$zero", "$hc", "44", I);
    instrucao_insert(instrucoes, "lw", "$zero", "$spc", "45", I);
    instrucao_insert(instrucoes, "lw", "$zero", "$pc", "46", I);
    instrucao_insert(instrucoes, "lw", "$zero", "$aux", "47", I);
    instrucao_insert(instrucoes, "lw", "$zero", "$gp", "48", I);
    instrucao_insert(instrucoes, "lw", "$zero", "$fp", "49", I);
    instrucao_insert(instrucoes, "lw", "$zero", "$sp", "50", I);
    instrucao_insert(instrucoes, "lw", "$zero", "$ra", "51", I);
    contInstrucoes = contInstrucoes + 31;
}
```

TROCA DE CONTEXTO - FUNÇÃO PARA
RESTAURAR REGISTRADORES

Troca de contexto - offset na MD

- 1 Memória de dados dividida em partições iguais de 150 posições
- 2 Flag ativa com a instrução Change_Offset, índice é o parâmetro
Instrução é chamada após ser realizado
- 3 salvamento dos regs do processo em execução

Código em Verilog

```
module Controle_Endereco_Memoria
(
    input Clock,
    input [3:0] Indice_Processo,
    input Change_Offset,
    output reg [31:0] Offset
);

always @(posedge Clock) begin
    if (Change_Offset == 1) begin
        case (Indice_Processo)
            4'b0000: Offset = 0;
            4'b0001: Offset = 150;
            4'b0010: Offset = 300;
            4'b0011: Offset = 450;
            4'b0100: Offset = 600;
            4'b0101: Offset = 750;
            4'b0110: Offset = 900;
            4'b0111: Offset = 1050;
            4'b1000: Offset = 1200;
            4'b1001: Offset = 1350;
            4'b1010: Offset = 1500;
            4'b1011: Offset = 1650;
            4'b1100: Offset = 1800;
            4'b1101: Offset = 1950;
            default: Offset = 0;
        endcase
    end
end
endmodule
```

Troca de contexto - interrupção

- 1 Ativa quando o limite do quantum é atingido
- 2 É realizado um jump para o endereço salvo durante a instrução jalr (volta para o SO)

Trecho da Unidade de Controle

```
always @ (OpCode)
begin
    if (quantum_end == 1) begin
        Reg_Dst <= 3'b100;
        Reg_Write <= 1;
        Jump_Register <= 2'b10;
        Entrada <= 2'b11;
        Halt <= 1;
        Ativa_PC <= 1;
        Conta_Halt <= 0;
        Quantum_flag <= 0;
    end else begin
```

Troca de contexto - offset na MI

Código em Verilog

- 1 Memória de instruções dividida em duas partições
- 2 Instrução jalr seta o Offset para 550
- 3 Sinais de Halt e de final do quantum setam o Offset para 0

```
module Controle_Offset_MI (  
    input Clock,  
    input Set_Offset,  
    input Halt,  
    input quantum_end,  
    output reg [31:0] saida  
);  
  
    initial begin  
        saida = 0;  
    end  
  
    // Atualiza OffsetAdd quando Set_Offset é ativado  
    always @(posedge Clock) begin  
        if (Set_Offset) begin  
            saida <= 550;  
        end  
        else begin  
            if (Halt || quantum_end) begin  
                saida <= 0;  
            end  
        end  
    end  
  
endmodule
```

Round Robin

Pseudocódigo - parte 1

```
exibir_LCD(15)

N = ler_entrada()
fila_processos = lista_vazia()

para i de 1 ate N faca
    programa = ler_entrada()
    adicionar programa em fila_processos
fim para

exit_loop = 0
ponteiro = 0

enquanto exit_loop == 0 faca
    processo_atual = fila_processos[ponteiro]

    // Executa processo_atual
    ler_entrada()
    carregar_programa(processo_atual)
    exibir_LCD(processo_atual)

    salvar_registradores()
    mudar_offset(processo_atual)
    restaurar_registradores()

    configurar_quantum()
    run()
```

Pseudocódigo - parte 2

```
salvar_registradores()
mudar_offset(0)
restaurar_registradores()
exibir_LCD(0)

// Verifica se terminou
halt = verificar_halt()

se halt == N entao
    exibir_LCD(12)
    remover processo_atual de fila_processos
    se fila_processos estiver vazia entao
        exit_loop = 1
        limpar_RAM()
    fim se
senao
    // se nao terminou, mantem o processo na fila
    ponteiro = (ponteiro + 1) mod tamanho(fila_processos)
fim se
fim enquanto
```

Execução com preempção - controle de halt

Código em Verilog

- 1 Módulo para contar quantidade de halts realizados
- 2 Armazena saída em registrador reservado
- 3 Ao final da execução dos programas na fila o registrador é zerado

```
module Controle_Halt
(
    input Clock,
    input Conta_Halt,
    output reg [31:0] saida // Contador de halts
);

    initial begin
        saida <= 0;
    end

    always @(posedge Clock) begin
        if (Conta_Halt) begin
            saida <= saida + 1;
        end
    end

endmodule
```

```
}else if(strcmp(quads->quad->var1, "checkHalt") == 0){
    instrucao_insert(instrucoes, "addi", quads->quad->resultado, "$hc", "0", I);
    contInstrucoes++;
}
```

Execução com preempção - limpeza da memória

Código em C-

- 1 Checa se a quantidade halts é igual a quantidade de processos na fila
- 2 Função para limpar RAM zera registradores essenciais para o funcionamento do SO na próxima chamada

```
halt = checkHalt();  
  
if (halt == qtdProcessos) {  
    setLcd(12);  
    exitLoop = 1;  
    clearRAM();  
}
```

Perguntas?